

# Governance Belongs to the Work

The Work-Class Governance Specification for Consequential Agentic Systems

Jeff Whatcott (Seampoint)

September 14, 2026

## Abstract

Agentic systems can authenticate, call approved tools and follow their instructions while still performing unauthorized business acts. Conventional controls often bind identity and tool access without preserving the authority, cumulative exposure, legal state and effect evidence of one specific work instance. This paper presents the Work-Class Governance Specification, a portable deterministic contract for bounded consequential work. A work class names the operations and objects in scope, required authority, typed constraints, shared limits, valid transitions and evidence of completion. A conforming consumer evaluates explicit artifacts, observations, events and state; the host commits the returned transition, enforces it at native dispatch and reports effect evidence through the same work identity. Candidate `seampoint.work-class/1.0.0-draft.2` contains 80 requirements and 425 frozen cases. TypeScript and Python reference consumers reproduce every case byte for byte, and four synthetic specimens apply the same contract in access, money, care and force. The evidence establishes a reviewable specification and conformance apparatus for the declared profiles. It does not qualify a production host, establish source-policy fidelity, prove operational benefit or supply facts that remain the host's responsibility.

**Keywords:** agentic AI governance; deterministic governance; work-class specification; business authority; conformance testing; shared reservations

*Open review preprint. Not posted to arXiv.*

**Candidate:** `seampoint.work-class/1.0.0-draft.2`

**Pin:** 5442ee3b

## 1 Why consequential agentic work is hard to govern

Organizations are handing work to prediction machines: systems that learn statistical relationships from data to predict information that is not directly available. Agrawal, Gans and Goldfarb describe modern artificial intelligence as prediction technology and gave their 2018 book the title *Prediction Machines*. [Prediction, Judgment, and Complexity](#), [Prediction Machines](#)

An agent that reads a request, chooses a tool and decides that a payment should go out can perform work once divided between a clerk and a controller. It cannot become the legal or institutional bearer of the authority under which that work is done. Under prevailing legal doctrines worldwide, artificial intelligence systems lack legal personhood and cannot possess legal rights or obligations. The Law Commission of England and Wales notes that current artificial intelligence systems lack legal personality and treats granting such personality as a possible future reform. [Law Commission discussion paper](#) International human rights law and comparative regulatory analyses confirm that

granting legal personhood to artificial intelligence is neither necessary nor desirable; legal systems require accountability to remain anchored in human beings, corporate principals and operating organizations. [Yousefi and Afshani](#); [Council of Europe Framework Convention on AI](#) In criminal and corporate law across jurisdictions including the United States, the European Union, the United Kingdom, China and Japan, liability presupposes voluntary human action (*actus reus*) and culpable mental intent (*mens rea*). Autonomous models possess neither consciousness nor moral agency, meaning legal accountability falls upon the humans and corporations who deploy and oversee them. [Sahu and Sarwan](#); [EU AI Act](#) As the Software Engineering Institute at Carnegie Mellon University observes in its research on autonomous systems, machines “fundamentally lack agency and cannot be held accountable for their actions.” [SEI CaTE Guidebook](#)

Governance must therefore let an agent choose and act while binding the authority for each decision, and accountability for its effect, to a named person or organization. That binding must survive the whole life of the work.

Suppose an accounts-payable agent processes a routine request received over email: update the banking details for an established vendor, Acme Industrial, and disburse 150,000 dollars for approved invoice #4810. The agent holds valid corporate API credentials, has tool access to the enterprise resource planning system, and is instructed by its system prompt to follow company policy. It locates the supplier in the vendor registry, inspects the invoice, and prepares an API call to transfer the funds to the newly provided routing number.

Every perimeter check passes. The agent’s cryptographic identity token is valid, the payment endpoint is on its allowed tool list, and its reasoning trace confirms that the invoice is approved and that paying suppliers on time is corporate policy. Yet whether this payment should actually occur remains entirely unsettled. The invoice was approved for Acme Industrial’s established bank account, not the new destination. The change request may have arrived in a phishing-driven vendor bank-change: an attacker impersonates a known supplier and sends fraudulent destination instructions. The confirmation may concern a supplier with a similar name. The person who confirmed the change may lack payment authority, or internal controls may bar the person who updates vendor details from releasing disbursements. The agent can execute a well-formed API call and still perform unauthorized work.

Removing payment access would prevent that payment and every legitimate payment in the same scope. The control has to stop this payment without stopping every payment. Suppose the organization requires an independent confirmation of new bank instructions, authority for the exact destination and invoice, and separation between confirmation and release. Each condition concerns this payment. The agent’s statement that it followed the policy cannot discharge any of them.

Dividing work across several agents does not solve the identity problem. One agent may receive the request, another confirm the destination, a third check the invoice and a fourth submit the transfer. Each can pass a local check while the required relationship remains unstated. Confirming the destination and approving the invoice does not establish that this invoice may be paid to this destination. The relationship belongs to the work instance and must survive handoff, retry and replacement of coordinating software.

Time adds another failure mode. A fact can be correct when checked and stale when used. A grant can be withdrawn between a preliminary check and dispatch. A confirmation service can be unavailable without the request becoming either valid or invalid. The contract has to say what absence means. Treating missing evidence as permission allows an unjustified action. Refusing every case with missing information adds restrictions the policy may not contain. A deterministic

contract keeps false, unavailable, stale and conflicting evidence distinct when their effects differ.

The native system creates a separate problem after permission. The native system is the service or system of record through which the effect is attempted and observed, distinct from the agent, the runtime and the connector between them. A request may time out after acceptance. An acknowledgement may mean receipt rather than settlement. A later record may concern another transaction. The record must distinguish a proposal, an authorization decision, a dispatch, a native acknowledgement, a settled effect and an unresolved outcome. A confident agent report cannot establish those relationships. A timestamp alone cannot show that two messages concern the same act.

Cumulative exposure makes per-call checks insufficient. Ten payments under a local threshold can exceed a daily cap. Two individually admissible reservations, whether they come from agents, from humans, or from both, can consume the same remaining capacity when they race. A copied balance or a fresh work-instance identifier cannot reset a shared budget. The policy must state whether it counts committed effects, reservations, unsettled exposure or some combination, and the state transition must change all affected partitions together.

Composition adds more ways to lose the governed identity. A choice has one selected successor, while a loop creates a sequence of occurrences under one bounded scope. A parallel split creates obligations that a join must track. A fan-out creates one occurrence for each member of a frozen set. A deadline depends on a declared clock event and activation anchor. If the runtime does not retain those identities, an event from one branch, iteration or object can discharge another.

Four recurring problems organize the proposal. The table uses the contract's review names. They are not synonyms for the operational failures above: non-convergence is uncovered source obligations, compounding error is lost meaning across steps, unbound evidence is an effect that is not bound to the instance, and delegation-chain accountability is authority that does not survive a participant change.

| Problem                         | Work-class response                                                                                               | Remaining boundary                                                                           |
|---------------------------------|-------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------|
| Non-convergence                 | Source correspondence, residue and review records expose uncovered obligations and unsupported provisions.        | They cannot prove that every relevant source requirement was found or interpreted correctly. |
| Compounding error               | Exact instance and occurrence identities, typed state and bounded routing preserve declared meaning across steps. | Only admitted state and declared profiles are governed.                                      |
| Unbound evidence                | Evidence records bind an operation, authority decision, dispatch and native effect to one work instance.          | The host must authenticate people and providers and report truthful native evidence.         |
| Delegation-chain accountability | The work instance carries authority and accountability across participant changes.                                | The organization must assign legitimate authority and enforce every dispatch path.           |

The controls surveyed here address parts of each problem. None of them bind all four to one persistent work instance.

A local permission, a local state transition or a local log can be correct while the end-to-end work is wrong. A stable, reviewable contract connecting authority, conditions, exposure, execution and evidence is missing. This standard defines that contract.

## 2 What existing controls accomplish, and where their contracts stop

Instructions, examples, system prompts, output guardrails, Constitutional AI, safety classifiers and RLHF evaluations help an agent understand an assignment and expose recurring errors. Isolation and restricted credentials reduce the systems an agent can reach. Those controls still help. They leave a model's reading of business policy outside the enforcement boundary. The workflow systems and emerging agent-control standards in this section sit beside those model-alignment tools.

Prompt injection illustrates the distinction. The United Kingdom's National Cyber Security Centre describes a language model's difficulty separating instructions from data inside a prompt and recommends controls outside the model for consequential actions. The recommendation supports placing decisive checks at a deterministic boundary. It does not define the business records, aggregate state or completion evidence that the boundary must bind. [NCSC prompt injection guidance](#)

Credential authorization answers who may call a service. It does not always answer whether the caller may perform this operation on this object with these facts. A payment agent can have a credential that permits transfers while the current invoice lacks the independent confirmation required by policy. A privileged-access agent can have a role that permits grants while the requester's separation-of-duty condition remains unmet. Identity names a principal. The decision needs the rest of the facts and their relationship to the exact operation.

Policy engines supply a second piece. Cedar evaluates whether a principal may take an action on a resource in a supplied context. Open Policy Agent accepts structured input, separates policy decisions from enforcement and can return structured decisions. These systems provide precedents for typed inputs and enforcement separation. The calling system still has to bind those inputs to the actual act and enforce the result. A policy engine cannot correct an application that checks one resource and dispatches another. [Cedar authorization](#), [Open Policy Agent](#)

Workflow systems supply a third piece. Business Process Model and Notation specifies process structure and execution semantics, including sequencing and control relationships. A workflow can put confirmation before payment or require several branches to complete before a join. A flow definition does not bind whether an approval covers the exact object, whether evidence is current or which native event proves completion. The work-class standard retains process structure and adds the missing subject, authority and effect relationships. [OMG BPMN 2.0.2](#)

Shared rate limiting shows how to coordinate cumulative exposure across instances. Envoy's global rate-limit design separates the local filter from an external service that coordinates a global limit across instances. The arrangement works only when relevant requests use the same correctly scoped counter and the service remains part of the request path. A local counter cannot stand in for a shared budget. A business cap has the same requirement, although its units and evidence come from policy. [Envoy global rate limiting](#)

Current agent-control efforts govern agents at a single call boundary. Microsoft's Agent Control Specification evaluates one intervention point over a host-supplied snapshot and is stateless by rule. The OWASP Agent Control Standard defines runtime hooks, telemetry conventions and an

agent bill of materials. Microsoft Agent Hooks names interception points with approval bound to a content hash. The OpenID AuthZEN Authorization API returns one decision for a subject, action and resource and places workflow and delegation out of scope. An IETF draft on attenuating authorization tokens carries per-invocation argument constraints and leaves completion tracking to the deployment. Each can be an enforcement target or carrier for a work-class artifact. None supplies the persistent work instance that carries authority, exposure, evidence and completion across calls and participants. [Microsoft Agent Control Specification](#), [OWASP Agent Control Standard](#), [AuthZEN Authorization API](#)

Autonomous weapons supply a published precedent. Arkin’s ethical governor placed a deterministic constraint check between a robot’s deliberation and a lethal action, configured from rules of engagement written in natural language. The Software Engineering Institute’s reference architecture for assuring ethical conduct in lethal autonomous weapons systems generalizes the pattern: formalized rules sit in one component, probabilistic components are wrapped by conduct governors, inputs and outputs are validated and traceability is retained. Its companion guidebook states the premise this paper starts from: machines “fundamentally lack agency and cannot be held accountable for their actions.” The work-class proposal applies the same separation to business work, with a persistent instance and explicit accountability for each admitted decision. Section 7 builds one of draft 2’s specimens from that architecture’s own rules-of-engagement example. [SEI reference architecture](#), [SEI CaTE guidebook](#)

The standards above serve different contracts. A policy engine answers an authorization query. A workflow system advances a graph. A rate-limit service coordinates a counter. A connector reports a native event. The problem is the join between their records. A work-class artifact gives those records one subject, one revision and one refusal and completion model.

### 3 Requirements for deterministic work-class governance

A common field list would not produce interoperability. Two products could accept the same fields and disagree about whether an observation is stale, whether a temporal boundary is included or whether an event completes the work. The standard therefore defines observable behavior, admitted inputs and refusal precedence.

First, exact identity. A proposal names the work-class digest, class revision, instance, occurrence, step, operation, interface, executor and all admitted fields. The runtime checks that the supplied proposal fields match the native request it will authorize. A retry with changed fields is a different subject. The first occurrence is explicit, such as `issue:1`; an omitted occurrence cannot become a new business act by default.

Second, exact value semantics. JSON numbers are excluded from policy-bearing artifacts. Decimal values use canonical strings and exact arithmetic. Typed grouping keys, sets and lists have declared element types and ordering. Temporal windows state endpoint inclusion, time zone, daylight-saving behavior and week or fiscal starts. A consumer cannot fill a missing rule from a language default.

Third, explicit authority. A credential, a reusable grant, an organizational attestation, a verification record and an instance decision are different records. The contract states which authority covers the operation, object, amount, stage and time. It checks the complete applicable set of records, independently selected roots, current revocation evidence and separation rules. Pending, rejected and deferred acts cannot establish acceptance. Authentication of the people and providers supplying those records remains a host obligation.

Fourth, cumulative exposure. An aggregate definition names its reducer, units, grouping, contribution mapping, qualifiers, window and bound. It states whether unsettled reservations count. Distinct cardinality requires the finite set needed to account for overlap. Shared reservations evaluate committed effects, pending exposure and the new contribution against one expected revision. All affected partitions change together or none change. A stale revision cannot silently retry against a new balance.

Fifth, stateful composition. A choice selects one declared successor. A loop has exact iteration limits and occurrence identities. A deadline records activation, clock and expiry. A parallel split creates obligations for the paths it actually activates. A fan-out freezes a typed object set before generating occurrences. A join waits for those exact obligations. Unsupported dynamic behavior is refused or excluded; the consumer never infers it.

Sixth, completion evidence. Dispatch is not completion. A native acknowledgement, settlement, failure, unknown outcome and disputed effect have different meanings. Completion evidence names the exact operation occurrence it covers. A foreign record cannot complete the work. An unknown outcome retains exposure and blocks unsafe redispach until the declared evidence and authorization resolve it.

Seventh, inspectable review. Readback accounts for every operative policy field in deterministic order. Source correspondence identifies both source obligations with no artifact provision and artifact provisions with no source support. Review records bind to exact subjects and distinguish source confirmation, policy decision and organizational authorization. These records make omissions visible. They do not claim to prove that an English source was understood correctly or that an external observation is true.

Eighth, conformance evidence. Each mechanical rule has a decisive case or property. Each external dependency has a documented boundary record with an owner, required control, evidence format, failure behavior, verification method and residual limitation. A fresh reviewer derives expected results from the normative rule and input before seeing consumer output. A second implementation consumes the contract without using the first implementation's semantic helpers. Mutation testing checks whether selected wrong implementations are detected. Reports name exact candidate, suite, implementation and dependency versions.

## 4 The proposed 1.0 standard

Candidate `seampoint.work-class/1.0.0-draft.2` is a closed, versioned contract. Its normative documents are self-contained. An implementer can read the candidate, resolve its schemas and run its cases without any other Seampoint document or product source.

The common representation uses [JSON Schema 2020-12](#) with a closed, bundled reference graph. Canonical serialization follows [RFC 8785](#) with the candidate's restricted input rules. Domain-separated SHA-256 digests identify complete subjects. The contract specifies duplicate-key rejection, invalid Unicode handling, decimal forms, list ordering, canonical-byte admission, references and version refusal. Source identifiers are inert. A reference cannot trigger a filesystem or network lookup.

A candidate manifest identifies each normative document and schema by path and hash. The manifest is versioned. A change to a normative document, schema or expected result creates a new candidate identity. Historical results stay attached to their original candidate. A consumer selects

a candidate explicitly and verifies the complete dependency closure before processing a request.

The protocol is language-neutral JSON lines. Its operations are `capabilities`, `validate`, `evaluate`, `aggregate-step`, `deploy`, `step`, `readback` and `check-evidence`. Each request carries the protocol identity, candidate identity, profile and claimed role. Each response carries a canonical decision or typed refusal. Stateful operations return receipts and complete next state. Malformed or inadmissible input leaves state and reservations unchanged. A policy refusal remains distinct from an invalid-event refusal.

The linear profile starts with a complete work-class definition. It names identity, revision, participants, objects, capabilities, authority requirements, steps, operations, field coordinates and linear relationships. Deployment binds a class revision to an instance and its authority and reservation records. In draft 2, deployment also requires three accepted review records bound to the exact definition, described in section 4.1. A `step` event names the exact occurrence and prior state revision. Successful completion requires declared native evidence. Duplicate events replay the established result. A foreign completion record, changed proposal or cross-instance event is refused.

The authority profile defines direct business authority. It separates executor credentials from grants and decision acts. It binds source, work class, envelope, proposal, operation, authority roots, credentials, acts, returns and observations to exact digests. It checks occurrence-time capacity and occupancy, reliance-time validity and revocation, complete checks, source selection and subject correspondence. Parameterized grants carry the restriction that one grant cannot be silently split across steps. The consumer checks the supplied facts; the host authenticates their sources and decides whether the organization has authorized the policy. When authority is not established, the refusal names each failed check, its subject, the requirement it tested and the reason it failed.

The aggregate profile supports COUNT, exact nonnegative SUM and distinct CARDINALITY. It accepts complete event snapshots or qualifying statistics under the declared query. It distinguishes unavailable, stale, conflicting and incomplete observations. It collapses byte-identical duplicates and refuses different reports for the same query. Rolling windows and normal calendar periods have explicit endpoint and time-zone rules. The bundled implementations pin IANA time-zone data to 2025a. Business-day calendars, signed netting and implicit currency conversion are excluded.

The shared-reservation profile defines a budget independently of any work instance. Its immutable definition fixes reducer, unit, grouping, window, bound, contribution mapping, permitted contributors and reservation policy. Exact work-class authorizations remain separate records, which avoids circular hashes. A reservation checks committed contributions, distinct unsettled reservations and the new contribution against one expected revision. Renewal replaces the prior receipt without adding its contribution twice. Settlement replaces reserved exposure with committed exposure once. Unexpected effects remain disputed or excess exposure. Timeout or revocation alone does not release capacity. Release requires affirmative evidence and authorization. A changed budget identity cannot erase outstanding exposure.

The composition profiles extend the same state model. Choice and bounded loops define the selected route, iteration limits and occurrence identities. Activity deadlines define activation, persisted expiry, clock events, late effects and completion before expiry. A deadline can also run from the completion of a named earlier step, so a policy's clock starts at the event that triggered the work rather than at the step that must meet it. Structured parallel work defines atomic obligation creation and joins over the paths actually activated. Frozen fan-out defines one occurrence per member of an admitted object set and retains the set even if the source later changes. Repeated expansion passes, dynamic fan-out, compensation, general cancellation and distributed execution

are excluded from 1.0.

The evidence profile defines deterministic readback, source correspondence, residue and review records. Readback includes every operative scalar leaf and empty collection in pointer order. Correspondence accounts for uncovered source obligations and unsupported artifact provisions. A review act binds an exact subject, reviewer role and candidate identity. Pending, rejected or deferred review acts cannot count as acceptance. Digests establish content identity. They do not authenticate an author.

A compact excerpt from an access-card work-class definition shows how operations, required authority, aggregate reservations and completion evidence are declared together:

```
{
  "schema": "seampoint.work-class/1.0.0-draft.2/definition",
  "identity": {
    "class_id": "corporate-access-card-issuance",
    "revision": "1"
  },
  "profiles": [
    "linear-core",
    "authority",
    "aggregates",
    "shared-reservations"
  ],
  "operations": [
    {
      "operation_id": "issue_badge",
      "interface": "physical-security-api",
      "native_action": "card_inventory.issue",
      "arguments_schema": {
        "type": "object",
        "required": ["employee_id", "card_id", "facility_code"],
        "properties": {
          "employee_id": { "type": "string" },
          "card_id": { "type": "string" },
          "facility_code": { "type": "string", "enum": ["HQ-01", "DC-02"] }
        },
        "additionalProperties": false
      },
      "authority_requirement": {
        "requirement_id": "req-facility-manager-approval",
        "grant_type": "FACILITY_ACCESS_DELEGATION",
        "required_roots": ["root-corp-security-ca"]
      },
      "reservation_claim": {
        "registry_id": "facility-capacity-registry",
        "budget_slot": "daily-card-issuance-cap",
        "charge_units": 1
      },
      "completion_evidence": {
        "evidence_type": "NATIVE_SYSTEM_RECEIPT",
        "provider_id": "card-controller-prov-01"
      }
    }
  ]
}
```

## 4.1 Attestation before deployment

A work-class definition enters service only after people have said, on the record, what it means and that it may run. The contract does not prescribe how an organization reaches those statements. It prescribes the records they produce, what each binds and what deployment refuses without them.

Three records are required, and they stay distinct even when one person holds several remits. A source confirmation says that the source obligations the correspondence accounts for are the organization's obligations as written. A policy decision says that the definition, read through its deterministic readback, is the policy to enforce. An organizational authorization says that this exact definition may be deployed for this instance under this candidate. Each binds an exact subject digest, names its actor and time, and counts only when accepted. The correspondence record sits beneath them: it gives every uncovered obligation and unsupported provision a residue row and refuses deployment while a row is pending. An excluded or unsupported row stays visible and bound, so the people who signed also signed the silence.

The contract fixes the canonical bytes of each act, which is what a signature would cover, but specifies no signature format, because signing infrastructure differs between organizations. The records establish something narrow: named people read one rendering and one stated silence and accepted them. The records do not show that those people read the source correctly or chose a wise policy. They show that no artifact runs without them and that a changed artifact requires new records.

## 5 Runtime placement and host obligations

The runtime belongs where a proposed consequential operation becomes dispatchable. Upstream agents and workflow systems can gather facts, choose among options and prepare proposals. The work-class runtime validates the selected class and authority, evaluates conditions and exposure, and returns a decision plus the complete next state. The host commits that state before dispatch, enforces the checked arguments, calls the native system and submits effect evidence under the declared occurrence identity.

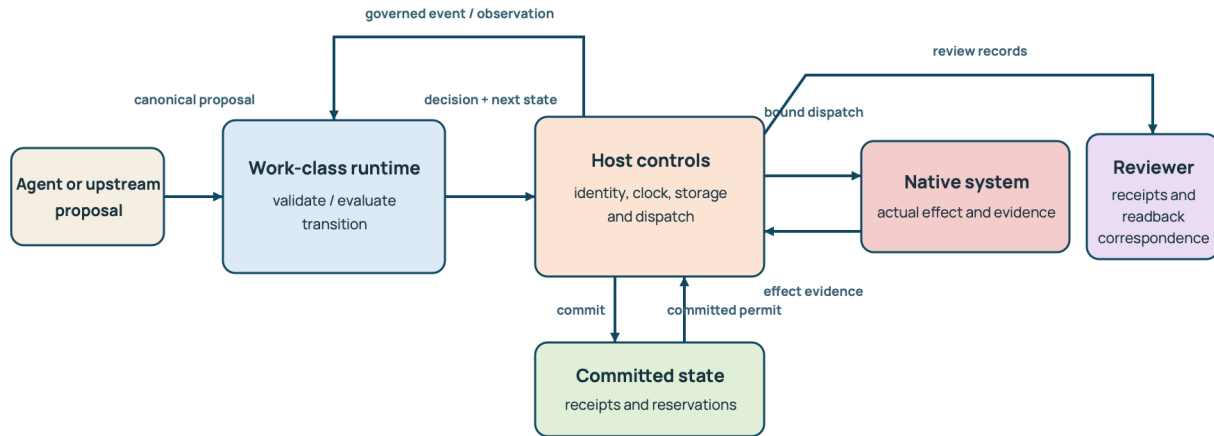


Figure 1: Figure 1. Runtime boundary and host responsibilities.

The diagram's arrows are contracts. A canonical proposal enters the runtime. The runtime returns

a decision and a state transition. The host must commit that transition and use the committed permit for native dispatch. Native evidence comes back through the host and becomes a governed event. Reviewers inspect the receipts, readback and correspondence. An ungoverned bypass path cannot claim enforcement by the runtime.

The interface between host, agent, runtime and native system follows a language-neutral JSON Lines protocol. A single operational occurrence advances through discrete steps:

```
{ "step": "PROPOSE", "instance_id": "inst-902", "occurrence": "disburse:1", "amount": "150.00", "account": "acct-new" }
{ "step": "EVALUATE", "runtime_verdict": "PERMITTED", "permit_id": "pmt-7712", "budget_charged": "150.00" }
{ "step": "COMMIT", "host_state": "PERSISTED", "status": "COMMITTED_PRE_DISPATCH" }
{ "step": "DISPATCH", "target": "wire-service", "permit_digest": "sha256:4a7e9..." }
{ "step": "NATIVE_EVIDENCE", "provider": "wire-service", "status": "SETTLED", "tx_id": "tx-99410" }
{ "step": "COMPLETE", "work_state": "COMPLETED", "reservation": "SETTLED_TO_COMMITTED" }
```

If a network fault occurs between DISPATCH and NATIVE\_EVIDENCE, the host records the occurrence status as UNKNOWN\_OUTCOME. The runtime retains the reservation, preventing other agents from consuming that capacity until out-of-band reconciliation either confirms settlement or clears the transaction.

## 5.1 Host integration and the enterprise baseline

An enterprise architect evaluating this boundary may reasonably ask whether deterministic governance simply displaces the hardest problems of distributed reliability onto the host environment. Adopting the standard requires deliberate engineering across policy authoring, runtime deployment and host adapters.

That investment is proportional to the stakes. Enterprises have largely confined autonomous agents to low-consequence advisory roles (such as drafting text, summarization or internal search) precisely because internal auditors, risk committees, cyber insurers and regulators cannot accept probabilistic models operating on consequential systems under bearer tokens and system prompts alone. The standard provides the deterministic guarantees required to unlock autonomous execution in high-stakes workflows.

Crucially, the standard does not require an enterprise to invent new distributed consensus protocols, deploy novel identity platforms or build custom database engines. High-consequence enterprise environments already operate and defend strict baselines to auditors: identity providers, relational databases with row-level locks, transactional queues and change-management ticketing.

The incremental work imposed by the standard is specific and bounded: 1. Intercepting the dispatch path so that native tools cannot be called without an active, committed permit. 2. Projecting existing enterprise facts (identity assertions, as-of database records, clock events) into canonical JSON inputs. 3. Persisting the returned state transition atomically before invoking the native system. 4. Feeding native execution receipts back to the runtime to settle reservations and close occurrences.

The table below maps each specification boundary directly to the enterprise controls that already discharge it. When a boundary is not met, the consumer's behavior is deterministic: it refuses, holds or retains exposure. It never permits by default.

| Boundary                                     | Existing enterprise control that discharges it    | Incremental host obligation                           | What the consumer does if missing          |
|----------------------------------------------|---------------------------------------------------|-------------------------------------------------------|--------------------------------------------|
| Identity and credentials                     | Identity providers (e.g. Okta, Entra ID)          | Project assertions into canonical credential records  | Refuses the operation                      |
| Organizational authorization and review acts | Change-management systems (e.g. ServiceNow, Jira) | Record subject digests through authenticated channels | Refuses deployment                         |
| Observation truth and freshness              | Systems of record with change-data-capture        | Supply revision and as-of timestamps                  | Withholds new work with declared reason    |
| Clock                                        | Network Time Protocol / disciplined time service  | Submit periodic monotonic clock events                | Holds; nothing expires                     |
| Durable state and event log                  | Relational databases or document stores           | Atomic write with expected-revision compare           | Refuses dispatch from uncommitted state    |
| One writer per instance and registry         | Database row locks or conditional writes          | Compare expected revision before commit               | Rejects stale transitions unchanged        |
| Native dispatch                              | API gateways and enterprise connectors            | Use permit digest as idempotency key                  | Retains unknown exposure; blocks redispach |
| Dispositions and stalls                      | Exception management queues and ticketing         | Submit administrative disposition acts                | Dispute stays charged and blocked          |

## 5.2 Why the host does not have to be perfect

Two technical properties ensure that the host environment does not require unattainable distributed guarantees:

First, permission is a snapshot with a declared lifetime. Every proposal-capable step declares a finite permit duration (`permit_seconds`). A successful proposal fixes the authority and observations on which it was judged until that expiry, and the organization explicitly accepts that staleness bound when it authorizes the work-class definition. The standard does not assume external facts never change; it forces the organization to record the acceptable staleness threshold on the record.

Second, the coordination required by the contract is per key, not global. The runtime requires one serialized writer for each work instance and each shared reservation registry. That is precisely what an ordinary relational row lock or a conditional write (such as DynamoDB conditional puts or PostgreSQL row-level locks) provides. The contract does not require distributed two-phase commits across disparate corporate systems.

The reference host included in the candidate is deliberately smaller than a production deployment. It serializes transitions within a single process, verifies expected revisions, and exercises two-client reservation contention. It demonstrates the transition contract and an architectural arrangement that makes it enforceable under test. It does not prove crash durability, network authentication,

connector idempotency, observation truth or distributed coordination. A deployment report must claim those properties only when its own controls and evidence establish them. A host conformance profile that a production binding can run against its own store, identity provider and connector is planned for a later specification candidate.

## 6 Evidence for specification candidate 1.0.0-draft.2

Every figure in this section comes from the draft 2 freeze record at the following candidate pin. Draft 1's results stay attached to draft 1.

`sha256:5442ee3b55675d849e388981487b806c0be29f686df6df8520b488508f86ceec`

The requirement inventory, generated from the normative text, has 80 requirements. The frozen suite has 425 cases. Of these, 296 carry the judgments of the expected-judgment contract; 295 have exact vectors, and one, a single clock event that expires 64 object occurrences across a 64-branch parallel block, is still pending. The remaining 129 are decisive controls and matrix cases outside that contract. Expected results, states and digests were derived from the cited text before either consumer ran, and the case index records that no implementation output was consulted. The runner evaluates nothing; it compares canonical bytes.

The TypeScript and Python consumers reproduce all 425 expected results byte for byte and agree with each other on every one, including refusals, receipts and complete next state. The four application specimens in section 7 pass under both. A self-contained export holding the specification, the corpus, both consumers, the runner and the specimens reruns the corpus and every specimen verifier with the same results and no link outside itself. A third implementation of the serialization and digest rules, written from the specification text alone without reference to either consumer, reproduced the evidence, subject and deployment-authorization digests of deployment case D2-008 byte for byte; the rules that give a record its identity are precise enough to be re-derived by a fresh reader.

One independent review round examined the draft 2 changes. Five reviewers covered the text, each consumer and the specimens, and a second agent in each area tried to refute every finding. They raised 34 findings: 5 did not survive, 28 were confirmed and 1 was left undecided. Fixes the text decided were made in whichever consumer was wrong, with new cases; questions the text did not decide went to the specification owner, whose decisions landed as three sentences and a schema change. Those corrections have not been independently reviewed, and the freeze record says so. Every confirmed finding concerned work that had already passed full verification, and three concerned behavior on which both consumers agreed with each other and not with the text. Agreement between consumers is weaker evidence of fidelity than the agreement figures suggest.

The two consumers and the serialized reference host are the transition kernel of a runtime. Given a definition, a state, an event and the host's evidence, they produce the decision, the receipts and the complete next state. A deployable runtime adds a durable store, an identity-provider adapter, observation providers, connector adapters, a clock submitter, a reservation registry service and an operator surface for stalls and disputes. Each is an integration against a control the deployer already has, and the boundary records say what each must guarantee.

Draft 2 lacks a mutation catalog, a generated-property suite and a closed requirement-coverage record. It provides less evidence that the corpus would detect an incorrect consumer, and it qualifies no role.

| Evidence item                                                                       | What it establishes                                                                                             | What it leaves to the deployer                                                    |
|-------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------|
| Normative documents and closed schemas                                              | The declared meaning and admitted representation of the candidate                                               | Whether an organization's policy is legitimate or complete                        |
| 425 frozen cases derived before either consumer ran TypeScript and Python agreement | Expected judgments for the named inputs<br>Independent consumers produce the same results for the frozen corpus | Behavior outside those inputs<br>Correctness of untested behavior or integrations |
| Four application specimens                                                          | The contract carries four unlike domains end to end                                                             | Fitness of any real policy in those domains                                       |
| One review round with its findings closed                                           | Defects a skeptical reader found were fixed or decided                                                          | Defects the round did not find, and the corrections made after it                 |
| Boundary records and the reference host                                             | Explicit host facts, owners, controls and failure behavior                                                      | Authenticity, truth and operation of those facts                                  |

## 7 Four application specimens

Reviewers of the first candidate asked whether one contract can hold across domains without a vocabulary of its own for each. Draft 2 answers with four specimens, one each in access, money, care and force. None is normative. Each applies the candidate to a synthetic policy, derives every expected result from the text before either consumer runs, and passes under both consumers byte for byte. Their review acts are labelled synthetic proxies: no person has confirmed these sources or decided these policies, and the records say so.

A specimen that exposes a general rule produces a contract change rather than a special case. The sepsis and rules-of-engagement specimens exposed sixteen issues. Draft 2 closed two with new constructs (a deadline running from an earlier step and named failed checks), resolved three with clarifying sentences, and marked two as unsupported; the rest are recorded for a later specification candidate.

### 7.1 Supplier payment

The supplier-payment specimen is section 1's example, built out. Its work class changes `supplier-17`'s destination account and then pays an invoice. Native evidence that the account change took effect supplies the payment proposal with exact prior-effect values, so a proposal naming the old account is refused before authority is evaluated. The permit retains the actor who confirmed the change and the actor who released the payment, and the class requires them to differ; when one person does both, the proposal is withheld. A shared budget counts committed and unsettled payments per supplier over a rolling 24 hours, with 800 dollars already counted against a limit of 1,000:

```
{
  "registry_id": "vendor-17-disbursements",
  "budget_revision": "1",
  "window": { "type": "ROLLING_24H", "seconds": 86400 },
  "capacity_limit": "1000.00",
  "committed_exposure": "800.00",
```

```

    "unsettled_reservations": [],
    "initial_available_capacity": "200.00"
}

```

Two clients each calculate a 150-dollar reservation against registry revision 1. A serialized host commits one, advancing the budget to revision 2 with 50 dollars of remaining capacity. The second proposal arrives stale and is refused with a revision conflict. Resubmitted against revision 2, it is withheld because 800 committed plus 150 pending plus 150 proposed is 1,100:

```

{
  "operation": "aggregate-step",
  "status": "REFUSED",
  "refusal": {
    "code": "BUDGET_WITHHELD",
    "detail": "BOUND_VIOLATED",
    "registry_id": "vendor-17-disbursements",
    "budget_revision": "2",
    "limit": "1000.00",
    "attempted_total": "1100.00",
    "available_capacity": "50.00"
  },
  "work_state": "UNMODIFIED"
}

```

The success trace then runs the payment the hard way. The payment is dispatched and acknowledged, and since an acknowledgement is not an effect, the occurrence stays open. The provider first reports that the outcome is unknown; the consumer keeps the 150 dollars charged and completes nothing. Only later evidence matching the dispatched request settles the reservation and completes the work. Each request carries its complete prior state, so a fresh consumer process admits it with no memory of earlier ones. The specimen does not show that a payment provider reports truthfully or that a host's store is durable. It shows that two independent consumers turn the supplied facts into the same decisions, and hold or refuse when a fact is missing.

## 7.2 Access card

Consider an automated physical-security workflow where an onboarding agent provisions building access badges for incoming contractors. The operation is linear and bounded: an authorized security officer delegates temporary issuance authority, the agent requests the badge from a badge-printing appliance, and the system waits for physical issuance confirmation from the controller.

The specimen exercises every boundary a consequential operation crosses: - Missing authority: when an onboarding agent attempts to issue a high-security data-center badge without a valid delegation from the facility manager, the runtime withholds the proposal (`AUTHORITY_REQUIRED`). - Parameter tampering: if an agent receives an authorized permit for `card-802` but submits dispatch arguments requesting `card-999`, or if an uncredentialed background worker attempts the dispatch, the runtime refuses the request as a binding mismatch. - Foreign native effects: if the badge controller reports a successful issuance event for a card identifier the runtime never authorized, the event is retained as a `FOREIGN_EFFECT` dispute, blocking that contractor's partition and keeping the legitimate occurrence open:

```

{
  "operation": "step",
  "status": "STEP",
  "disposition": "DISPUTED",

```

```

"classification": "FOREIGN",
"reason_codes": [
  "FOREIGN_EFFECT",
  "BUDGET_EFFECT_DISPUTED",
  "UNEXPECTED_EFFECT"
],
"blocked_partitions": [
  {
    "anchor": "cards-per-beneficiary",
    "key": [{"type_ref": "b.identity", "value": "employee-7"}]
  }
]
}

```

- Idempotent replay: resubmitting an identical completion event for an already-issued badge replays the stored transition without issuing duplicate credentials or advancing counters.

### 7.3 Inpatient sepsis surveillance

Consider an inpatient clinical setting where an automated surveillance agent continuously monitors patient vital signs and lab results. When the model detects patterns indicative of sepsis, it triggers an alert. The work-class contract does not judge whether the diagnostic prediction is clinically accurate; it governs the consequential care protocol that must follow: ordering emergency screening labs under a hospital standing order, paging the attending physician, escalating to a rapid-response team if the page goes unanswered, and enforcing that an antibiotic order is signed by a licensed physician before medication is dispensed.

The policy is synthetic and is not clinical guidance. Its hour-one antibiotic window runs from the alert however late the antibiotic step activates, a rule earlier candidates could not express and draft 2 now does through an ancestor-anchored deadline definition:

```

{
  "id": "antibiotic-window",
  "step_id": "order-antibiotic",
  "clock_source": "ward-clock",
  "due": {
    "kind": "ELAPSED_FROM_ANCESTOR",
    "value": "60",
    "unit": "MINUTE",
    "anchor_step_id": "raise-alert"
  },
  "boundary": "AT_OR_AFTER",
  "expiry_target": null
}

```

When an agent proposes antibiotic administration before the attending physician signs the clinical order, the runtime withholds the permit and identifies the missing decision gate:

```

{
  "operation": "step",
  "status": "STEP",
  "disposition": "WITHHELD",
  "authority_status": "REFUSED",
  "authority_code": "GATE_REQUIRED",
  "reason_codes": ["AUTHORITY_REQUIRED", "GATE_REQUIRED"],
  "permit": null
}

```

Two approximations remain for a later specification candidate: the window runs from the alert's delivery rather than its clinical recognition, and the attending's fifteen minutes run from the page step's activation rather than from the page.

## 7.4 Rules of engagement

The fourth specimen evaluates whether deterministic governance holds in safety-critical domains where an action cannot be rolled back or compensated after execution. Built from the tactical rules-of-engagement scenario in section 2.3.1 of the Software Engineering Institute's reference architecture, the specimen governs target engagement proposals by an autonomous weapon system. It is synthetic, adds no targeting capability, makes no claim about any weapon system or the lawfulness of any engagement, and implies no endorsement by the Institute or its authors.

The architecture requires two independent layers of human authority before the weapon system can fire: a reusable standing grant issued by a commander defining approved engagement criteria, and a per-instance tactical decision by a human operator on the exact track. Its permit expires automatically after 120 seconds. A shared budget allows three engagements per mission per hour, counting those whose effect is not yet established.

When the weapon system's autonomous track classifier evaluates a target but classification confidence falls below the commanded threshold (e.g. 0.82 against a required 0.90 bound), the runtime withholds the engagement permit. Draft 2 adds the requirement that the refusal identify the exact failed check:

```
{
  "operation": "step",
  "status": "STEP",
  "disposition": "WITHHELD",
  "reason_codes": [
    "AUTHORITY_REQUIRED",
    "AUTHORITY_NOT_ESTABLISHED",
    "CONDITION_VIOLATED"
  ],
  "authority_failed_checks": [
    {
      "requirement_ref": "roe-confidence-at-or-above-bound",
      "purpose": "CONDITION",
      "reason": "CONDITION_VIOLATED"
    }
  ],
  "permit": null
}
```

The specimen also marks where draft 2 stops: a disputed engagement has no attributable human exit, a required escalation leaves no runtime record, and there is no operator stop.

## 8 How to use and review the standard

An implementer begins with a candidate manifest and declares a role. The manifest, normative documents, bundled schemas and examples supply the contract. The suite supplies separately derived expectations. Language, data structures, database and policy engine are implementation choices. Refusal precedence, canonical records, state transitions and supported profile behavior are role obligations.

An implementation may use Cedar for a policy query, Open Policy Agent for structured evaluation, BPMN tooling for graph management, a database for reservation state or an existing connector for native dispatch. The integration must preserve the work-class subject across those parts. A policy engine cannot remove native-argument binding. A database counter cannot decide which events the policy counts. A connector acknowledgement cannot become completion without the declared evidence. A workflow transition cannot bypass a current authority or reservation check.

A first integration should choose one consequential operation whose policy and effect can be stated exactly. The access-card example in the candidate is suitable because it has a bounded object, an executor, an authority envelope, explicit conditions and native completion evidence. The implementer should identify the class revision, every operation occurrence, authority roots, observation providers, clock source, shared budgets, storage transaction, connector identity and every bypass path. The resulting deployment boundary record becomes part of the implementation's review package.

The specification deliberately separates authoring from runtime governance. The standard prescribes no specific authoring tool, pipeline or translation method; any process that yields an admitted definition with accepted correspondence conforms. An organization may author definitions through human policy committees, developer tooling or automated assistants, provided the output satisfies the normative schemas and candidate manifest.

The core authoring responsibility is accounting for policy boundaries: mapping source requirements to typed operations, declaring authority requirements and aggregate budgets, and generating inspectable readback. Where a source requirement cannot be expressed in the declared profile, or where an artifact provision lacks source support, the authoring process must record that gap in the correspondence residue for human review. This shifts review cost from runtime monitoring to definition release: a work class is audited and attested once per revision, rather than re-evaluated on every transaction.

A reviewer tests four questions in order. Can two readers derive one result from the same complete inputs? Does the declared profile preserve the distinctions that change authority, obligations, admissibility, execution or completion? Can an independent implementer proceed without proprietary semantic code? Does the suite detect a materially wrong consumer? Findings should cite the candidate, paragraph, input, governing rule, expected result and practical effect.

Early reviewers raised a composition objection: actions permitted separately can combine into a prohibited result. Shared reservations and the included composition profiles govern the relationships they declare. Interactions outside those declared relationships remain outside the guarantee.

The candidate's runner is a comparison tool; it evaluates no policy. It executes trusted local programs with bounded output, per-request timeouts and subprocess cleanup. It treats missing cases, skips, crashes, timeouts, malformed responses and duplicate results as failures. The controls are not a hostile-code sandbox. Fixture data contains no real credentials.

A conformance report names the candidate identity, contract hash, suite hash, implementation hash, claimed roles, cases, controls, dependencies, reproduction commands, failures and limitations. A report with all cases passing but a missing coverage control is not a qualified report. A report with a passing semantic role and an unqualified host obligation must state both facts. Old results remain attached to old candidates.

During the open review phase, outside readers report findings rather than submit specification text or source code. A finding should identify a concrete requirement, the disputed normative text, a

decisive example, the expected judgment and the practical effect. Seampoint will not accept outside contributions until it publishes a contribution policy, governance process and contributor agreement. A later contribution process will bind every accepted change to the Community Specification License 1.0.

External technical reviewers are asked to assess precision, sufficiency, independent implementability and whether the suite detects material violations. They are not asked to certify an organization's policy or a production deployment. Reviewers can work from a fresh export, run both consumers offline and inspect the hashes in the package manifest. The questionnaire asks them to separate a specification gap, an implementation defect, an evidence gap and a host obligation.

The complete review candidate, conformance suite, reference consumers, licenses and reproduction instructions are available in the [reviewer repository](#).

## 9 Claims and limits

This specification makes a narrow engineering claim: a portable deterministic contract can make the relationship between an admitted policy, authority, proposed action, cumulative exposure, state transition and recorded effect inspectable and testable for a declared profile. Draft 2 supplies a concrete contract, two consumers, a serialized reference host, four application specimens and evidence for the named inputs. No role is qualified.

The evidence does not establish that the standard covers every consequential business process, nor does it show that a work-class runtime reduces incidents, lowers review costs, or improves operator acceptance compared with a carefully integrated conventional system. Those claims require field studies. Nor does it prove that a native system executed an act, that an observation was true, that a person authenticated a credential, or that a distributed store remained linearizable; those are host and deployment properties. The specimens' policies and review acts are synthetic. One review round examined the draft 2 changes, and the corrections made after it have not been reviewed. Draft 2 leaves a disputed reservation and a blocked shared-budget anchor blocked; it names no clearance operation. Human disposition of a disputed effect, business-day boundaries, a host conformance profile and pre-approved materiality thresholds are deferred to a later specification candidate. The application specimens are technical examples, not legal, financial, clinical or defense advice.

The candidate does not certify an authoring system's ability to interpret arbitrary English policy. Any authoring method can produce an admitted artifact. Readback and correspondence help reviewers find omissions and unsupported provisions. They do not prove that the artifact faithfully captured a source policy. Seampoint's authoring tools are not prerequisites for implementing the candidate contract.

Copyright © 2026 Seampoint LLC. The normative specification is available under the Community Specification License 1.0. The conformance apparatus and reference consumers are available under Apache 2.0. This paper is available under Creative Commons Attribution 4.0. Seampoint's current claim analysis identifies no patent claim necessary to implement the specification; the specification license governs any Seampoint-controlled Necessary Claims within its recorded scope. No license grants a right to imply Seampoint sponsorship, certification or endorsement. Earlier candidates keep their recorded results under their own pins; those results say nothing about draft 2.

Seampoint develops commercial authoring and implementation technology and provides advisory services for consequential agentic systems. Those interests may benefit from adoption of the speci-

fication. The specification does not require that technology, and the licenses do not grant rights to technology outside their recorded scope.

Give an independent team one bounded operation, its exact policy, its native effect and its host facts. Ask that team to produce a conforming artifact and runtime, then run the same cases and inspect the same records. If the team must ask what a field means, the contract needs work. If the team can implement the contract but the host cannot supply a required fact, the boundary record should say so. A standard earns its place by making both situations visible before an agent is allowed to act.